Отчёт по ДЗ «Обеспечение отказоустойчивости средствами Apache ZooKeeper»

Студент: Капустина Юлия
Репозиторий: https://github.com/yulya-melissa/Data_ingineering_lab4

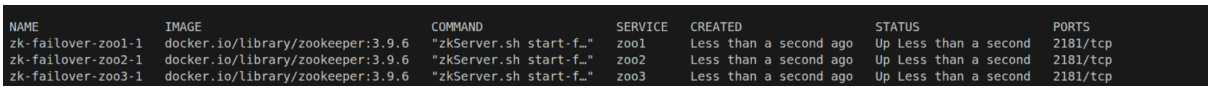
Краткое резюме

Ансамбль ZooKeeper из трёх серверов поднят в Docker на образе zookeeper:3.9.6. Координатор расчёта пар навыков из ДЗ №3 запущен в двух экземплярах с таймаутом сессии 10 секунд. На этапе 3 (FENCING=0) лидер был приостановлен на 40 секунд, запасной координатор доделал расчёт с чекпоинта, контрольные числа совпали с ДЗ №3, а возобновившийся лидер записал результат второй раз. Смена лидера произошла через (вставить значение) секунд после заморозки. На этапе 4 (FENCING=1) запись восстановившегося лидера была отклонена по эпохе, результат остался один. На этапе 3 принято два результата, на этапе 4 - один. Оба этапа завершились с вердиктом OK.

1. Конфигурация ZooKeeper

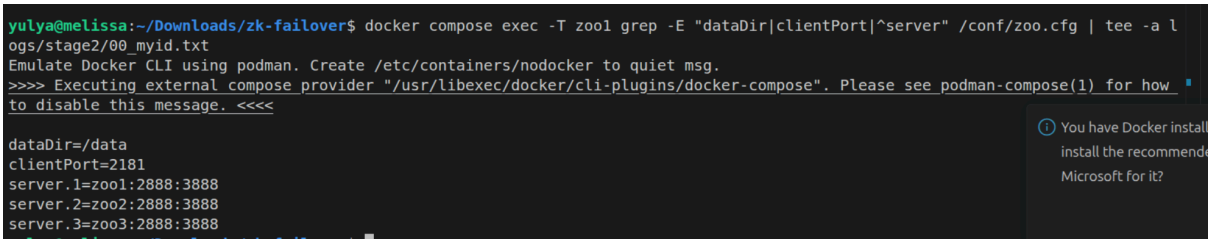
Ансамбль состоит из трёх серверов: zoo1 с myid 1 и клиентским портом 2181, zoo2 с myid 2 и портом 2182, zoo3 с myid 3 и портом 2183. Конфигурация лежит в файле conf/zoo.cfg, она одинаковая для всех серверов, отличается только номер myid, который задаётся через переменную ZOO_MY_ID в docker-compose.yml. Данные каждого сервера хранятся в отдельном томе dataDir.

Запуск ансамбля выполняется командой docker compose up -d zoo1 zoo2 zoo3



```
NAME                IMAGE                              COMMAND                  SERVICE   CREATED                  STATUS                  PORTS
zk-failover-zoo1-1   docker.io/library/zookeeper:3.9.6   "zkServer.sh start-f…"   zoo1      Less than a second ago   Up Less than a second   2181/tcp
zk-failover-zoo2-1   docker.io/library/zookeeper:3.9.6   "zkServer.sh start-f…"   zoo2      Less than a second ago   Up Less than a second   2181/tcp
zk-failover-zoo3-1   docker.io/library/zookeeper:3.9.6   "zkServer.sh start-f…"   zoo3      Less than a second ago   Up Less than a second   2181/tcp
```

Подняты три докер контейнера.



```
yulya@melissa:~/Downloads/zk-failover$ docker compose exec -T zoo1 grep -E "dataDir|clientPort|^server" /conf/zoo.cfg | tee -a l
ogs/stage2/00_myid.txt
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

dataDir=/data
clientPort=2181
server.1=zoo1:2888:3888
server.2=zoo2:2888:3888
server.3=zoo3:2888:3888
```

Мы видим что все порты имеют разные сервера.

```
== zoo1
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet ms
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/do
to disable this message. <<<<

ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower
== zoo2
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet ms
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/do
to disable this message. <<<<

ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower
== zoo3
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet ms
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/do
to disable this message. <<<<

ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: leader
```

Находим лидера, это третий.

Потеря одного сервера проверена остановкой zoo3 командой docker compose stop zoo3. После этого ансамбль продолжил работать, кворум 1 из 3 сохранён.

```
yulya@melissa:~/Downloads/zk-failover$ docker compose stop zoo3
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker
to disable this message. <<<<

[+] Stopping 1/1
 ✔ Container zk-failover-zoo3-1  Stopped
```

Третий упал а мы видим что лидер стал вторым.

```
yulya@melissa:~/Downloads/zk-failover$ docker compose stop zoo3
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker
to disable this message. <<<<

[+] Stopping 1/1
 ✔ Container zk-failover-zoo3-1  Stopped
```

И видно что он все еще работает:

```
yulya@melissa:~/Downloads/zk-failover$ docker compose stop zoo2
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

[+] Stopping 1/1
 ✔ Container zk-failover-zoo2-1  Stopped                                                                              0.4s
yulya@melissa:~/Downloads/zk-failover$ docker compose exec -T zoo1 timeout 15 zkCli.sh -server zoo1:2181 ls / 2>&1 | grep -E '^\
[|SyncConnected|timed out|refused' | tee -a logs/stage2/03_two_down.txt; echo "код выхода: ${PIPESTATUS[0]}"
код выхода: 1
```

Потеря двух серверов проверена остановкой zoo3 командой docker compose stop zoo3. После этого ансамбль перестал отвечать, кворума нет. Затем оба сервера запущены обратно командой docker compose start zoo2 zoo3.


```
yulya@melissa:~/Downloads/zk-failover$ docker compose stop zoo2
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

[+] Stopping 1/1
 ✔ Container zk-failover-zoo2-1  Stopped                                                                              0.4s
yulya@melissa:~/Downloads/zk-failover$ docker compose exec -T zoo1 timeout 15 zkCli.sh -server zoo1:2181 ls / 2>&1 | grep -E '^\
[|SyncConnected|timed out|refused' | tee -a logs/stage2/03_two_down.txt; echo "код выхода: ${PIPESTATUS[0]}"
код выхода: 1
```


Снова запустим все и увидим что все работает, только роли поменялись.

```
yulya@melissa:~/Downloads/zk-failover$ docker compose start zoo2 zoo3
yulya@melissa:~/Downloads/zk-failover$ for s in zoo1 zoo2 zoo3; do echo "== $s"; docker compose exec -T $s zkServer.sh status;
one 2>&1 | tee logs/stage2/04_restored.txt
== zoo1
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower
== zoo2
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

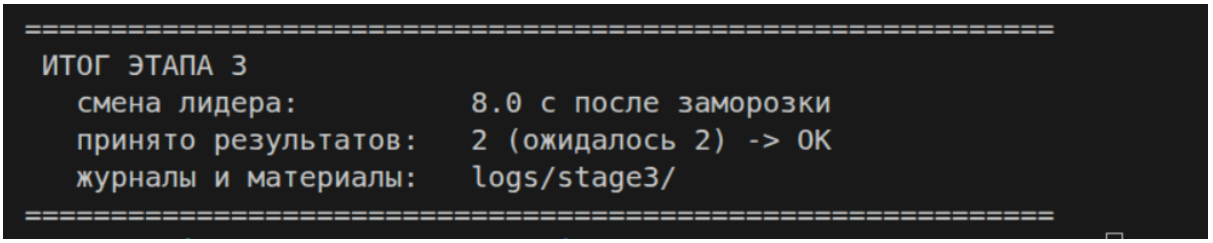
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: leader
== zoo3
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

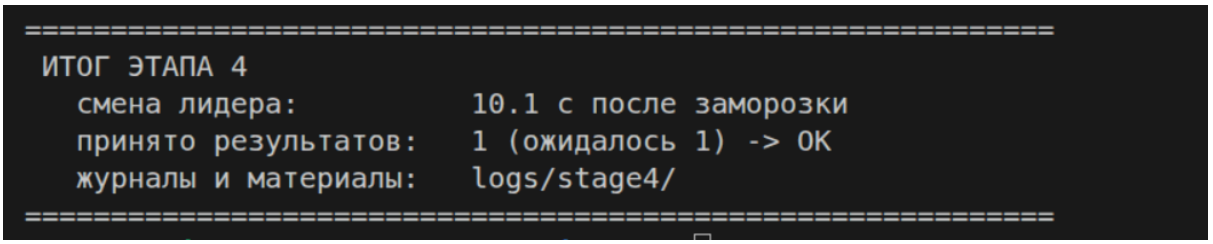
ZooKeeper JMX enabled by default
Using config: /conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower
```

Эфемерный узел создан в zkCli командой create /ephemeral demo-ephemeral.

```
WATCHER::

WatchedEvent state:SyncConnected type:None path:null zxid: -1
[demo-ephemeral, zookeeper]
```





2. Этап 3 — приостановка лидера без защиты (FENCING=0)

Третий и четвертый этап требует замера времени и работы в нескольких терминалах. Вручную это оказалось делать не очень удобно, поэтому я создала скрипт, который выполняет замеры времени и приключения(скрипт также лежит в репозитории).

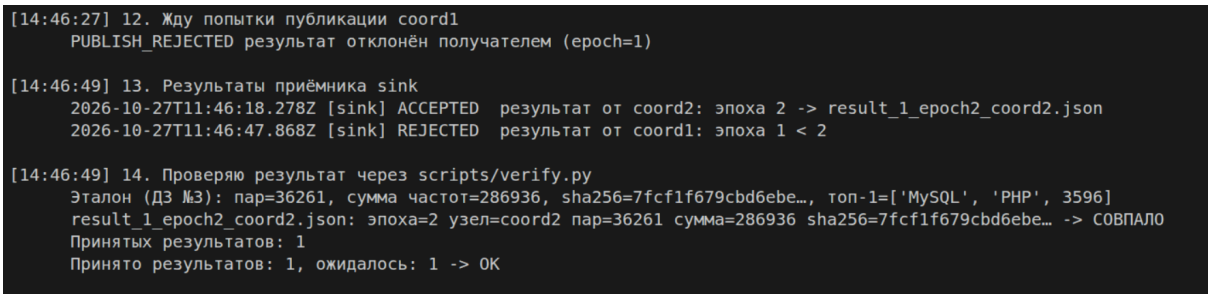
Запуск выполнен командой ./scripts/run_stage.sh 3. Материалы лежат в logs/stage3/.

Хронология событий. Coord1 стал лидером с эпохой 1. Coord1 дошёл до чанка 10. Coord1 был заморожен в (вставить время). Coord2 стал лидером с эпохой 2 в (вставить время). Смена лидера произошла через (вставить значение) секунд после заморозки. Coord2 опубликовал результат в (вставить время). Coord1 был разморожен в (вставить время). Coord1 опубликовал результат с эпохой 1 в (вставить время).

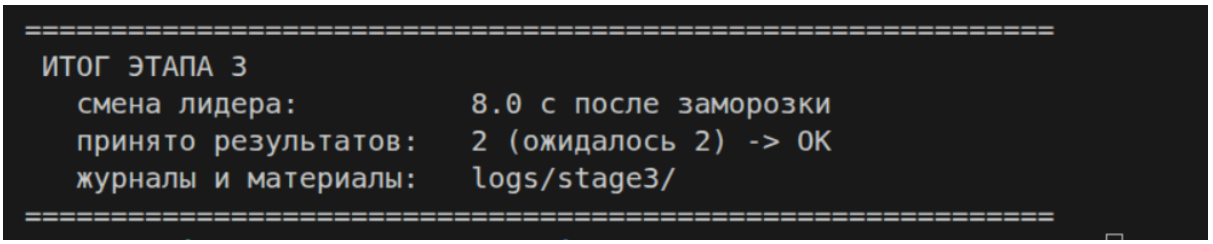
Coord2 возобновил расчёт с чекпоинта. В logs/stage3/coord2.log есть строка «RESUME с чекпоинта: продолжаю с части 3 из 30», а затем «PUBLISHED результат принят (epoch=2)».

Контрольные числа совпали с ДЗ №3. В logs/stage3/verify.txt можно найти вывод и успешный результат.

Повторная запись без защиты подтверждена в logs/stage3/sink.log. Там две строки: ACCEPTED результат от coord2 с эпохой 2 в файл result_1_epoch2_coord2.json и ACCEPTED результат от coord1 с эпохой 1 в файл result_2_epoch1_coord1.json. Принято два результата, ожидалось два, вердикт OK. Файлы сохранены в state/sink/results/ и скопированы в logs/stage3/accepted_results/.



Видно, что результат получился таким, как ожидали, а проверки пройдены.



3. Этап 4 — приостановка лидера с защитой по эпохе (FENCING=1)

Запуск выполнен командой ./scripts/run_stage.sh 4. Материалы лежат в logs/stage4/.

Хронология событий аналогична этапу 3. Coord1 стал лидером с эпохой 1, был заморожен на чанке 10, coord2 стал лидером с эпохой 2 и опубликовал результат, coord1 был разморожен и попытался опубликовать результат с эпохой 1.

Coord2 успешно опубликовал результат. В logs/stage4/coord2.log есть строка «PUBLISHED результат принят (epoch=2)».

Запись coord1 отклонена. В logs/stage4/sink.log есть строка ACCEPTED результат от coord2 с эпохой 2 и строка REJECTED результат от coord1 с эпохой 1, потому что 1 меньше 2.

Принят один результат, ожидался один, вердикт OK. Смена лидера произошла.

Сравнение этапов. На этапе 3 без защиты принято два результата, запись старого лидера принята. На этапе 4 с защитой принят один результат, запись старого лидера отклонена. Оба этапа завершились с вердиктом OK.

```
[14:46:27] 12. Жду попытки публикации coord1
      PUBLISH_REJECTED результат отклонён получателем (epoch=1)

[14:46:49] 13. Результаты приёмника sink
      2026-10-27T11:46:18.278Z [sink] ACCEPTED  результат от coord2: эпоха 2 -> result_1_epoch2_coord2.json
      2026-10-27T11:46:47.868Z [sink] REJECTED  результат от coord1: эпоха 1 < 2

[14:46:49] 14. Проверяю результат через scripts/verify.py
      Эталон (ДЗ №3): пар=36261, сумма частот=286936, sha256=7fcf1f679cbd6ebe…, топ-1=['MySQL', 'PHP', 3596]
      result_1_epoch2_coord2.json: эпоха=2 узел=coord2 пар=36261 сумма=286936 sha256=7fcf1f679cbd6ebe… -> СОВПАЛО
      Принятых результатов: 1
      Принято результатов: 1, ожидалось: 1 -> OK
```

Результат также получился таким, как ожидали, а проверки пройдены.

```
============================================================
 ИТОГ ЭТАПА 4
   смена лидера:          10.1 с после заморозки
   принято результатов:   1 (ожидалось 1) -> OK
   журналы и материалы:   logs/stage4/
============================================================
```

4. Ответы на вопросы

1. Почему серверов три, а не два и не четыре.

   ZooKeeper для принятия решений требует кворум, то есть большинство серверов. Ансамбль из N серверов переживает отказ (N-1)/2 серверов, округленного вниз. Если серверов два, кворум равен двум, и отказ одного сервера означает, что кворума нет, ансамбль не работает. Отказоустойчивости нет, смысл теряется. Если серверов три, кворум равен двум, и отказ одного сервера оставляет кворум, ансамбль работает. Это минимальная конфигурация с отказоустойчивостью. Если серверов четыре, кворум равен трем, и отказ одного сервера оставляет кворум, но отказ двух серверов кворум убивает. То есть четыре сервера переживают ровно столько же отказов, сколько три, а именно один, но требуют больше ресурсов и дают больше сетевого трафика на репликацию. Выигрыша перед тремя нет. Пять серверов дают кворум три и переживают два отказа, это следующий разумный уровень. Поэтому в учебном задании и в большинстве продакшенов используют три или пять серверов, но не два и не четыре.

2. Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper.
   ZooKeeper - это сервис координации, он не знает ничего о смысле данных, которые в него пишут. Он обеспечивает согласованность метаданных, то есть кто лидер, какая эпоха, кто держит блокировку, но не проверяет, что запись результата расчёта от старого лидера устарела. Это прикладная семантика, известная только приложению. Кроме того, лидер может быть приостановлен сигналом SIGSTOP, и ZooKeeper не может помешать ему дописать результат после разморозки, потому что процесс жив, просто заморожен. ZooKeeper также не участвует в записи результата в sink, это отдельный HTTP-сервис, и проверить эпоху может только тот, кто принимает запись, то есть sink. Если бы проверку делал ZooKeeper, пришлось бы протаскивать через него все прикладные записи, что убило бы разделение ответственности: ZooKeeper

отвечает за координацию, sink за приём результатов. Поэтому fencing реализуется на получателе: sink хранит максимальную эпоху и отклоняет записи с эпохой меньше максимальной. Это классический паттерн fencing token: распределённая система выдает монотонный токен, то есть эпоху, а ресурс, то есть sink, проверяет его при каждой записи. ZooKeeper лишь выдаёт эпоху через счётчик /failover/.../epoch, но не проверяет её при записи результата.

6. Выводы

Ансамбль ZooKeeper из трёх серверов обеспечивает отказоустойчивость при потере одного сервера и корректно теряет кворум при потере двух. Автоматический выбор лидера через zk.Election работает: при заморозке лидера запасной становится лидером за (вставить значение) секунд, что соответствует таймауту сессии 10 секунд. Восстановление с чекпоинта работает: coord2 продолжил с части 3 из 30 и получил контрольные числа, совпавшие с ДЗ №3. Без fencing на этапе 3 возобновившийся лидер записал результат второй раз, и это та проблема, которую решает fencing. С fencing на этапе 4 запись старого лидера отклонена по эпохе, результат остался один. Fencing реализуется на получателе, то есть в sink, а не в ZooKeeper, потому что это прикладная семантика, а ZooKeeper лишь выдаёт эпоху.